

Reviewer Pack — Secant Variety Dimension Lower Bounds Disjointness Communica...

Entry b35b32763c26 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 13:42:40 UTC

Secant Variety Dimension Lower Bounds Disjointness Communication Complexity

Entry ID: b35b32763c26 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 13:42:40 UTC

1. Conjecture

Field A (mathematical branch): ALGEBRAIC_GEOMETRY_OF_SECANT_VARIETIES **Field B** (complexity object): COMMUNICATION_COMPLEXITY

Statement:

For a Boolean matrix M representing the disjointness function on n -bit inputs, let $\tau(M)$ be the dimension of the secant variety of the set of points $\{ (x, y) \mid M[x][y] = 1 \}$ in the projective space $P^{\{n-1\}} \times P^{\{n-1\}}$. Then, the randomized communication complexity $R(M)$ satisfies $R(M) \geq \Omega(\tau(M))$ with $\tau(\text{DISJ}_n) = \Omega(n)$.

Rationale (proposer's reasoning):

Secant varieties capture the geometric complexity of point configurations, and their dimension reflects the minimal number of rank-1 tensors needed to span the matrix. By linking this algebraic invariant to communication complexity, we isolate a novel geometric obstruction to efficient protocols for disjointness.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 75544fed81d95ec0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        # Eliminate entries below pivot
        for j in range(i+1, n):
            factor = -A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] += factor * A[i][k]
            b[j] += factor * b[i]

    # Back-substitute to find solution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def rank(matrix):
    A = [row[:] for row in matrix]
    b = [0] * len(matrix)
    sol = gaussian_elimination(A, b)
    non_zero_count = sum(1 for entry in sol if entry != 0)
    return non_zero_count

def secant_dimension(M):
    n = len(M)
    points = [(i, j) for i in range(n) for j in range(i+1, n) if M[i][j] == 1]
```

```

if not points:
    return 0

# Construct the secant ideal
secant_ideal = []
for (x1, y1), (x2, y2) in itertools.combinations(points, 2):
    A = [[M[x1][y], M[x2][y]] for x, y in points]
    b = [0] * len(A)
    secant_ideal.append(gaussian_elimination(A, b))

# Compute the rank of the matrix spanned by the secant ideal
return rank(secant_ideal)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    M = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        M[i][i] = 0

    secant_dim = secant_dimension(M)
    R_M = n * (n - 1) // 2 # Lower bound on communication complexity for DISJ_n
    conjecture_holds = R_M >= 0.5 * secant_dim
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "secant_dimension",
        "metric_value": secant_dim,
        "instances_tested": n * (n - 1) // 2,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results) and support_fraction >= 0.8:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6c34eb15.py", line 91, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6c34eb15.py", line 72, in run_trial
    secant_dim = secant_dimension(M)
                ^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6c34eb15.py", line 60, in secant_dimension
    secant_ideal.append(gaussian_elimination(A, b))
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6c34eb15.py", line 33, in gaussian_elimination
    A[j][k] += factor * A[i][k]
    ~~~~^
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during gaussian_elimination with index error, preventing dimension calculation | next: Debug gaussian_elimination implementation to handle matrix indexing properly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112941
2	preregistration	ollama_remote	qwen3:8b	0	0	24182
3	novelty	ollama_remote	qwen3:8b	0	0	20503
4	novelty	ollama_remote	qwen3:8b	0	0	16443
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15635
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11234
7	judge	ollama_remote	qwen3:8b	0	0	13951

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 214890 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/b35b32763c26.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b35b32763c26.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b35b32763c26.tar.gz` (if generated)